

DAA Lab - Experiment 1

Implement the iterative and recursive Binary Search Tree and compare their performance

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement a Binary Search Tree (BST) with iterative and recursive search.
- Measure and compare their performance.
- Understand time and space trade-offs between the two approaches.

- A Binary Search Tree (BST) is a binary tree where:
 - Left child $<$ root
 - Right child $>$ root
- Recursive search uses function calls and call stack.
- Iterative search uses loops without recursion.
- In a balanced BST, both have $O(\log n)$ time complexity.

Algorithm: Recursive BST Search

Recursive_BST_Search(root, key)

- 1 If $\text{root} = \text{NULL}$, return NULL
- 2 If $\text{key} = \text{root.key}$, return root
- 3 If $\text{key} < \text{root.key}$, search in left subtree
- 4 Else, search in right subtree

Algorithm: Iterative BST Search

Iterative_BST_Search(root, key)

- ① Set current = root
- ② While current $\neq$ NULL:
 - If key = current.key, return current
 - Else if key < current.key, current = current.left
 - Else, current = current.right
- ③ Return NULL

C Code - BST Implementation (ONLY FOR REFERENCE)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct Node {
    int key;
    struct Node *left, *right;
};

struct Node* newNode(int item) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

struct Node* insert(struct Node* root, int key) {
    if (root == NULL) return newNode(key);
    if (key < root->key)
        root->left = insert(root->left, key);
    else if (key > root->key)
        root->right = insert(root->right, key);
    return root;
}
```

Recursive Search (ONLY FOR REFERENCE)

```
struct Node* searchRecursive(struct Node* root, int key) {  
    if (root == NULL || root->key == key)  
        return root;  
    if (key < root->key)  
        return searchRecursive(root->left, key);  
    return searchRecursive(root->right, key);  
}
```

Iterative Search (ONLY FOR REFERENCE)

```
struct Node* searchIterative(struct Node* root, int key) {  
    while (root != NULL) {  
        if (key == root->key)  
            return root;  
        else if (key < root->key)  
            root = root->left;  
        else  
            root = root->right;  
    }  
    return NULL;  
}
```


Main Function (ONLY FOR REFERENCE)

```
int main() {
    struct Node* root = NULL;
    int n, i, key;
    clock_t start, end;
    double tRec, tIter;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        root = insert(root, val);
    }
    printf("Enter key to search: ");
    scanf("%d", &key);

    start = clock();
    struct Node* r = searchRecursive(root, key);
    end = clock();
    tRec = ((double)(end - start)) / CLOCKS_PER_SEC;

    start = clock();
    struct Node* it = searchIterative(root, key);
```

Sample Input/Output

Input:

Number of nodes: 7

50 30 20 40 70 60 80

Key: 60

Output:

Recursive: Found

Iterative: Found

Time Recursive: 0.002 ms

Time Iterative: 0.001 ms

Complexity Analysis

- Time Complexity:
 - Best: $O(1)$
 - Average: $O(\log n)$
 - Worst: $O(n)$ (skewed tree)
- Space Complexity:
 - Recursive: $O(h)$
 - Iterative: $O(1)$

Observation Table

Nodes	Iterative Time (ms)	Recursive Time (ms)
100	0.002	0.003
500	0.005	0.008
1000	0.009	0.014

Comparison

- Iterative is more memory efficient.
- Recursive is easier to write and understand.
- Both have same time complexity for balanced trees.

Viva Questions

- 1 State the BST property.
- 2 Which approach uses more memory and why?
- 3 When does BST search become $O(n)$?
- 4 Why might iterative be preferred in deep trees?

Conclusion

- Both approaches yield same results.
- Iterative saves memory, recursive is simpler in code.
- Choice depends on application and constraints.

References

- Cormen et al., *Introduction to Algorithms*, MIT Press.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.